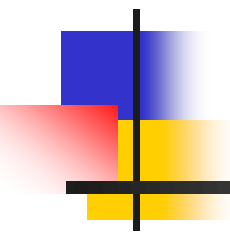
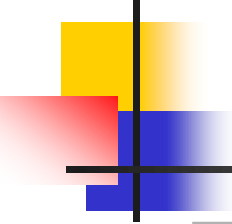


DATA STRUCTURES AND ALGORITHMS



Lecture Notes 5.1

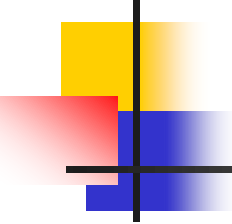
Pradit Pitaksathienkul



ROAD MAP

- TREES

- Implementation of Trees
- Tree Traversals
- Binary Trees
- **The Search Tree ADT-Binary SearchTrees**
- AVL Trees



Binary Search Trees

Each node in tree stores an item

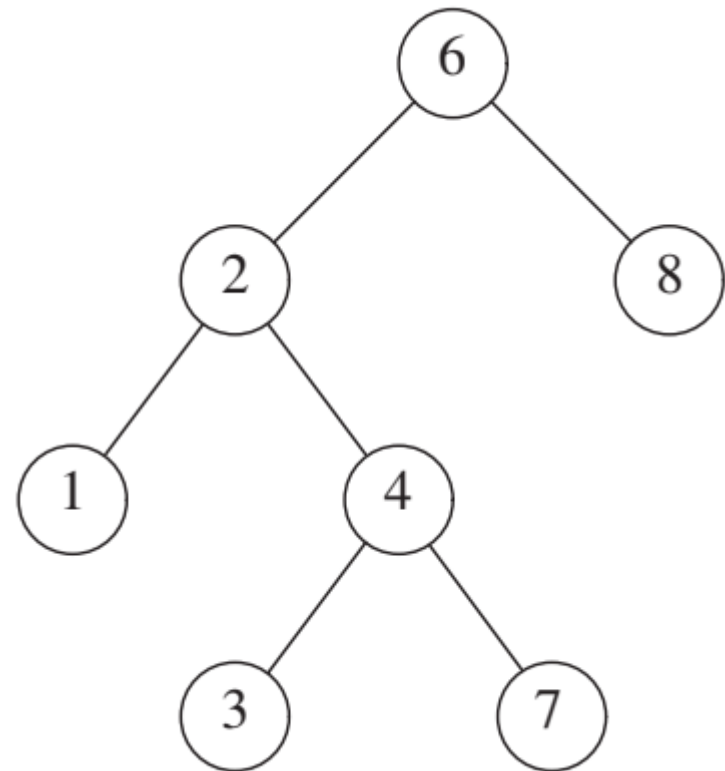
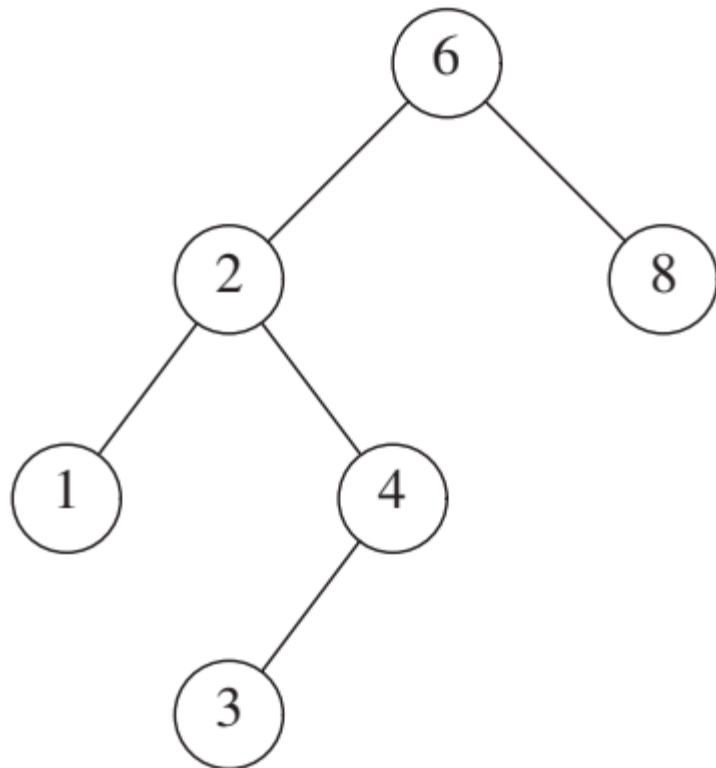
- items are integers and distinct
- when has no left or right child node, mean left or right point to None.

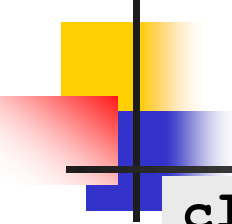
Properties

- A binary tree
- for each node x
 - values at left subtree are smaller
 - values at right subtree are larger than the value of x

Binary Search Trees

Which one of the trees below is binary search tree ?





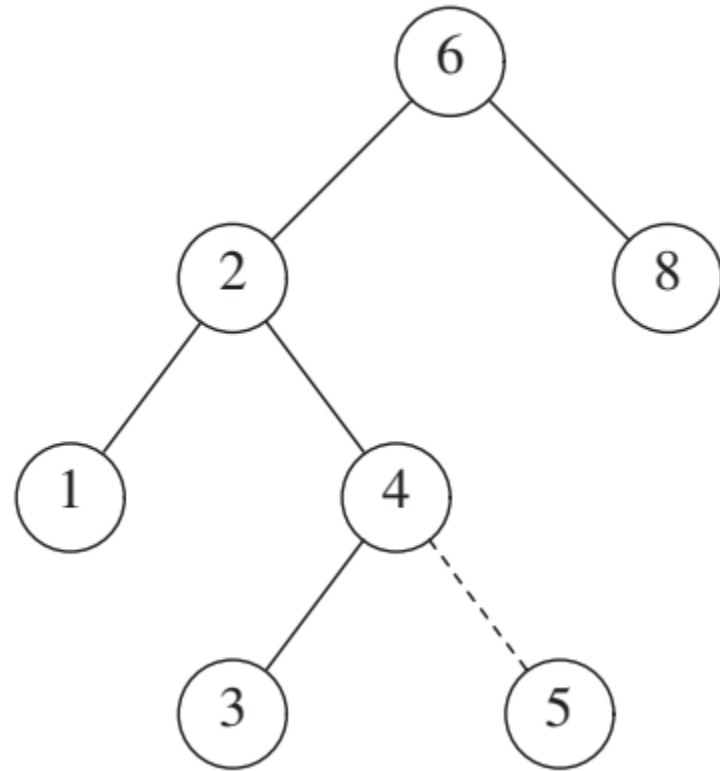
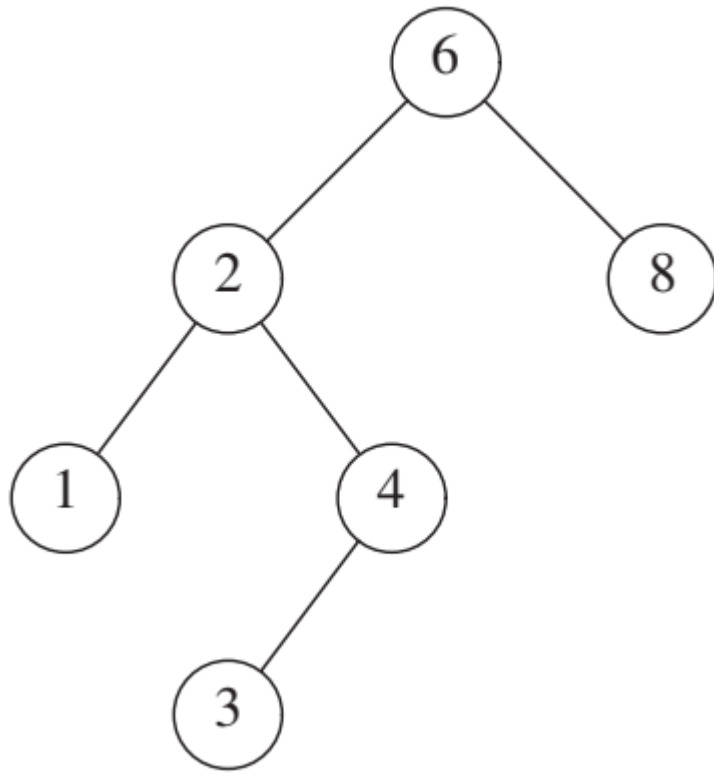
Binary Search Trees

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, value):
        if self.root is None:
            self.root = Node(value)
        else:
            self._insert_recursive(self.root, value)
```

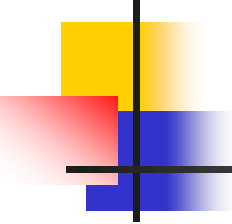
insert into a binary search tree





insert into a binary search tree

```
def _insert_recursive(self, current_node, value):
    if value < current_node.value:
        if current_node.left is None:
            current_node.left = Node(value)
        else:
            self._insert_recursive(current_node.left, value)
    elif value > current_node.value:
        if current_node.right is None:
            current_node.right = Node(value)
        else:
            self._insert_recursive(current_node.right, value)
    else:
        pass
```



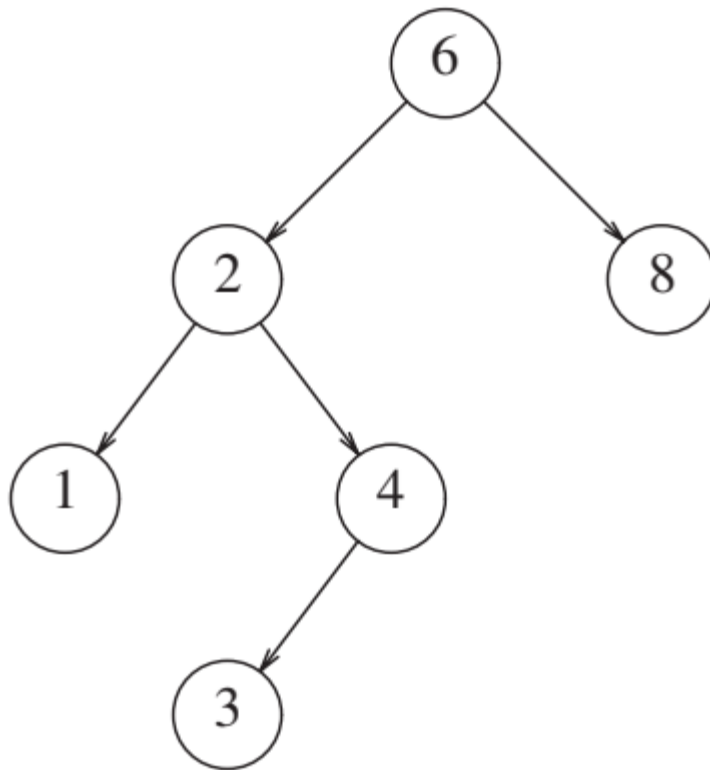
insert into a binary search tree

We write recursive function.

- Follow the definition as we learned with Tree.
- the single underscore function like as `_insert_recursive()` mean this function is used internal in class, we will not write this single underscore function at main program.
- we interested in the steps in each round or each iteration , **what statements are executed** under **the if-else condition** that is checked as **true**.

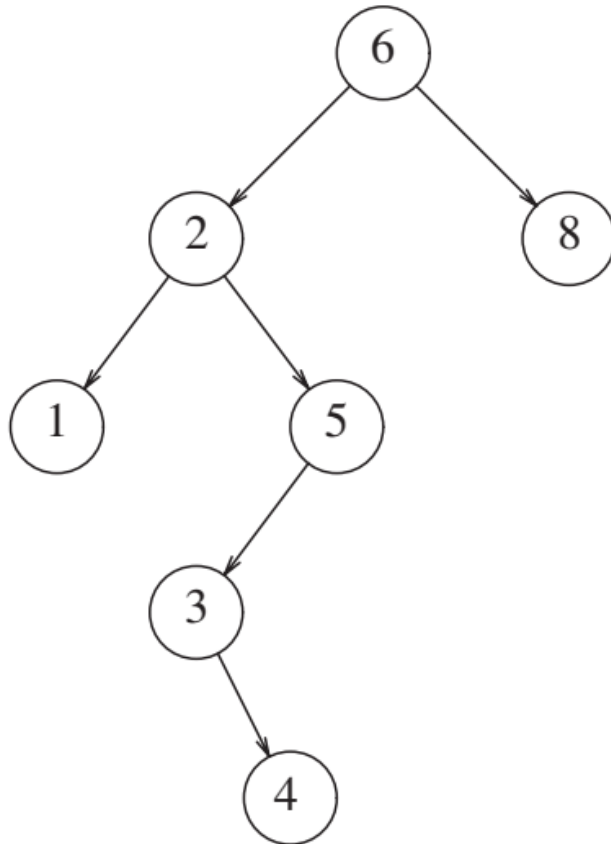
Remove from a binary search tree

When the node has one child. (4)



Remove from a binary search tree

When the node has two children. (2)



Remove from a binary search tree

```
def delete(self, value):
    self.root = self._delete_recursive(self.root, value)

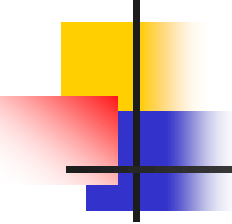
def _delete_recursive(self, current_node, value):
    if current_node is None:
        return current_node

    if value < current_node.value:
        current_node.left =
            self._delete_recursive(current_node.left, value)
```

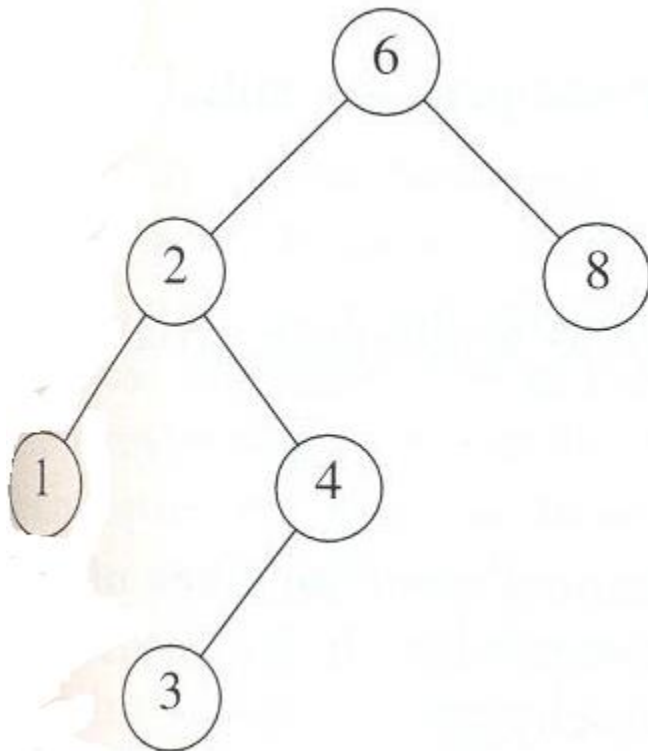
```
elif value > current_node.value:
    current_node.right =
        self._delete_recursive(current_node.right, value)
else:
    if current_node.left is None:
        return current_node.right
    elif current_node.right is None:
        return current_node.left

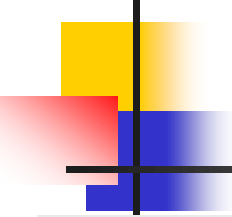
    temp_node =
self._min_value_node(current_node.right)
    current_node.value = temp_node.value
    current_node.right =
        self._delete_recursive(current_node.right,
temp_node.value)

return current_node
```



find





Find min

```
/* Find min value node in the tree - Return the current  
node that minimum */
```

```
def _min_value_node(self, node):  
    current = node  
    while current.left is not None:  
        current = current.left  
    return current
```

- What else to find the max value?